

# Project Management System (PMS)

Backend API — Complete Technical Documentation

**Organization:** Apparatus Solutions

**Version:** 1.0.0

**Generated:** June 11, 2026

**Stack:** Django 6.0 · Django REST Framework · PostgreSQL · Docker · Jenkins

# Table of Contents

---

- 1. Introduction
- 2. Architecture
  - 2.1 System Overview
  - 2.2 Technology Stack
  - 2.3 Project Structure
  - 2.4 API Modules & Role-Based Access
  - 2.5 Key Features
- 3. Database Design
  - 3.1 Entity Relationship Overview
  - 3.2 Core Tables
  - 3.3 Relationships & Constraints
- 4. Deployment
  - 4.1 Environment Configuration
  - 4.2 Local Development
  - 4.3 Docker Deployment
  - 4.4 Production (Jenkins CI/CD)
  - 4.5 Scheduled Jobs & Integrations

# 1. Introduction

The **Project Management System (PMS)** is a role-based backend API designed to streamline project planning, milestone tracking, task assignment, time logging, and team collaboration for Apparatus Solutions. It serves as the central data and business logic layer for the Nexus frontend application.

## Purpose

PMS enables organizations to manage the full project lifecycle—from creating projects and milestones to assigning tasks, tracking employee work hours, handling deadline change approvals, and delivering real-time notifications. The system supports three distinct user roles with tailored permissions and dashboards.

## User Roles

| Role                  | Responsibilities                                                                               |
|-----------------------|------------------------------------------------------------------------------------------------|
| ADMIN                 | Full system control: user management, project CRUD, deadline approvals, exports, AI assistant. |
| BA (Business Analyst) | Project & milestone management, task creation/assignment, deadline change requests.            |
| EMPLOYEE              | Task execution, self-created tasks, time tracking (start/pause/stop), personal dashboard.      |

## Key Capabilities

- JWT-based authentication with first-login OTP and password reset flows
- RESTful API with OpenAPI/Swagger documentation (drf-spectacular)
- Weighted progress tracking based on estimated hours
- Automatic timer auto-stop at configurable cutoff (default 8 PM)
- File attachments for projects, milestones, and tasks
- Email notifications for admin actions and deadline alerts
- Admin AI assistant powered by Ollama (read-only context)
- Internal service APIs for attendance system integration
- CSV/Excel export for projects, milestones, and tasks

## 2. Architecture

### 2.1 System Overview

PMS follows a **three-tier architecture**: a React-based frontend (Nexus) communicates with the Django REST API backend, which persists data in PostgreSQL (production) or SQLite (local development). Media files are stored on the filesystem. Optional Ollama LLM service provides AI assistance for administrators.

| Layer              | Components                                                                     |
|--------------------|--------------------------------------------------------------------------------|
| Presentation Layer | Nexus Frontend (React) — dashboards, task boards, auth UI                      |
| Application Layer  | Django REST Framework API — business logic, permissions, serializers           |
| Data Layer         | PostgreSQL (prod) / SQLite (dev) — relational data; filesystem — media uploads |
| External Services  | SMTP (email), Ollama (AI), Attendance Service (internal notifications)         |

### 2.2 Technology Stack

| Category          | Technology                                                      |
|-------------------|-----------------------------------------------------------------|
| Runtime           | Python 3.13                                                     |
| Web Framework     | Django 6.0.4                                                    |
| API Framework     | Django REST Framework 3.16.1                                    |
| Authentication    | djangorestframework-simplejwt (JWT Bearer tokens)               |
| API Documentation | drf-spectacular (OpenAPI 3 / Swagger / ReDoc)                   |
| Database          | PostgreSQL (production) via dj-database-url; SQLite (local dev) |
| CORS              | django-cors-headers                                             |
| Static Files      | WhiteNoise (production static serving)                          |
| WSGI Server       | Gunicorn (production) / Django runserver (development)          |
| Containerization  | Docker + Docker Compose                                         |
| CI/CD             | Jenkins Pipeline                                                |
| AI Integration    | Ollama (gemma4:e2b model)                                       |

### 2.3 Project Structure

```
Project_management/
├── pms/
│   ├── manage.py # Django management entry point
│   ├── requirements.txt # Python dependencies
│   └── Dockerfile # Development container
```

- Dockerfile.prod # Production container (Gunicorn)
- docker-compose.yml # Local dev orchestration
- docker-compose.prod.yml # Production compose config
- Jenkinsfile # CI/CD pipeline definition
- .env.example # Environment variable template
- API\_MASTER\_GUIDE.md # API usage guide
- schema.yaml # OpenAPI schema export
- pms/ # Django project config
- ■■■ settings.py # App settings & env config
- ■■■ urls.py # Root URL routing
- ■■■ wsgi.py / asgi.py # WSGI/ASGI entry points
- ■■■ bundled\_media.py # Bundled documentation serving
- pms\_api/ # Main application module
- models.py # Database models (9 entities)
- views.py # API view classes & viewsets
- serializers.py # Request/response serializers
- urls.py # API v1 route definitions
- permissions.py # Role-based access control
- authentication.py # Flexible JWT authentication
- export\_views.py # Admin CSV/Excel exports
- ollama\_client.py # AI assistant integration
- timer\_state.py # Work timer management
- timer\_auto\_stop.py # Scheduled timer auto-stop
- progress.py # Weighted progress calculation
- migrations/ # Database migration history (24 migrations)
- management/commands/ # CLI commands (superuser, notifications, sync)

## 2.4 API Modules & Role-Based Access

| Module         | Base Path               | Access              | Description                                           |
|----------------|-------------------------|---------------------|-------------------------------------------------------|
| Authentication | /api/v1/auth/*          | All                 | Login, refresh, first-login OTP, forgot password, /me |
| Users          | /api/v1/users/          | Admin               | CRUD for Admin, BA, Employee accounts                 |
| Projects       | /api/v1/projects/       | Admin, BA           | Project lifecycle; Admin-only deadline edits          |
| Milestones     | /api/v1/milestones/     | Admin, BA           | Milestone CRUD with auto milestone_no                 |
| Tasks          | /api/v1/tasks/          | Admin, BA, Employee | Task assignment, self-created tasks, status updates   |
| Files          | /api/v1/files/          | Authenticated       | Upload attachments to project/milestone/task          |
| Notifications  | /api/v1/notifications/  | Authenticated       | In-app notification inbox                             |
| Dashboards     | /api/v1/*/dashboard     | Role-specific       | Admin, BA, Employee dashboard metrics                 |
| Work Tracking  | /api/v1/work-tracking   | Employee            | Timer start/pause/stop for tasks                      |
| Admin Exports  | /api/v1/admin/exports/* | Admin               | Export projects, milestones, tasks                    |
| Admin AI       | /api/v1/admin/ai/*      | Admin               | Ollama-powered read-only Q&A                          |
| Internal APIs  | /api/v1/internal/*      | Service Token       | Attendance service integration                        |

## 2.5 Key Features

- **Time Tracking:** Employees start/pause/stop timers on tasks. Active sessions auto-stop at 8 PM (configurable) with grace period.
- **Progress Calculation:** Weighted progress uses task estimated\_hours; completed tasks count full weight.
- **Deadline Workflow:** BA requests deadline changes; Admin approves/rejects via ProjectDeadlineChangeRequest.
- **First Login Flow:** New users receive OTP email, verify, set password, then access the system.
- **Service Integration:** Attendance service pushes leave notifications via PMS\_SERVICE\_TOKEN bearer auth.

### 3. Database Design

#### 3.1 Entity Relationship Overview

The database consists of **9 core entities** built on Django's built-in User model. All domain models extend an abstract **TimeStampedModel** (created\_at, updated\_at). Relationships follow a hierarchical structure: Project → Milestone → Task, with TimeLog sessions attached to tasks and UserProfile extending User for role management.

```
auth_user (Django built-in)
■
■■■ 1:1 ■■ UserProfile (role, status, department, tech_stack, OTP fields)
■
■■■ 1:N ■■ Project (created_by) ■■ 1:N ■■ Milestone ■■ 1:N ■■ Task
■ ■ ■
■ ■ ■■ assigned_to → auth_user
■ ■ ■■ created_by → auth_user
■ ■ ■■ supervisor → auth_user
■ ■ ■■ 1:N ■■ TimeLog (user, duration)
■ ■
■ ■■ 1:N ■■ FileAttachment
■
■■■ 1:N ■■ Notification (user, ref_type, ref_id, details JSON)
■■■ 1:N ■■ ProjectDeadlineChangeRequest (project, requested_by, reviewed_by)
■■■ 1:N ■■ FileAttachment (uploaded_by)
```

#### 3.2 Core Tables

**UserProfile** — Extended user profile (1:1 with auth\_user)

| Column             | Type           | Constraint | Description                 |
|--------------------|----------------|------------|-----------------------------|
| user_id            | FK → auth_user | OneToOne   | Linked Django user          |
| role               | VARCHAR(20)    | —          | ADMIN   BA   EMPLOYEE       |
| status             | VARCHAR(20)    | —          | ACTIVE   INACTIVE           |
| experience_level   | VARCHAR(20)    | —          | JUNIOR   SENIOR             |
| department         | VARCHAR(100)   | —          | Free-text department        |
| tech_stack         | VARCHAR(100)   | —          | Technology expertise        |
| password_set       | BOOLEAN        | —          | First-login completion flag |
| first_login_otp    | VARCHAR(6)     | —          | OTP for account activation  |
| password_reset_otp | VARCHAR(6)     | —          | OTP for password reset      |

**Project** — Top-level project container

| Column        | Type           | Constraint | Description                                       |
|---------------|----------------|------------|---------------------------------------------------|
| name          | VARCHAR(255)   | —          | Project name                                      |
| description   | TEXT           | —          | Project description                               |
| created_by_id | FK → auth_user | PROTECT    | Creator (Admin/BA)                                |
| start_date    | DATE           | —          | Project start                                     |
| deadline      | DATE           | —          | Project deadline (Admin-only edit)                |
| status        | VARCHAR(20)    | —          | PLANNED   ACTIVE   COMPLETED   DELAYED   ARCHIVED |
| document      | FILE           | —          | Uploaded project document                         |

**Milestone** — Project phases with auto-incremented milestone\_no

| Column                | Type           | Constraint         | Description                                     |
|-----------------------|----------------|--------------------|-------------------------------------------------|
| project_id            | FK → Project   | CASCADE            | Parent project                                  |
| milestone_no          | INTEGER        | UNIQUE per project | Auto-generated sequence                         |
| name                  | VARCHAR(255)   | —                  | Milestone name                                  |
| start_date / end_date | DATE           | —                  | end_date immutable after creation               |
| status                | VARCHAR(20)    | —                  | NOT_STARTED   IN_PROGRESS   COMPLETED   DELAYED |
| created_by_id         | FK → auth_user | PROTECT            | Creator                                         |



### Task — Work items assignable to employees

| Column                   | Type           | Constraint        | Description                                              |
|--------------------------|----------------|-------------------|----------------------------------------------------------|
| project_id               | FK → Project   | CASCADE, nullable | Optional direct project link                             |
| milestone_id             | FK → Milestone | CASCADE, nullable | Parent milestone                                         |
| title                    | VARCHAR(255)   | —                 | Task title                                               |
| assigned_to_id           | FK → auth_user | SET_NULL          | Assigned employee                                        |
| created_by_id            | FK → auth_user | PROTECT           | Task creator                                             |
| supervisor_id            | FK → auth_user | SET_NULL          | Supervisor for self-created tasks                        |
| is_self_created          | BOOLEAN        | —                 | Employee-created task flag                               |
| status                   | VARCHAR(20)    | —                 | NOT_STARTED   IN_PROGRESS   PAUSED   COMPLETED   DELAYED |
| priority                 | VARCHAR(20)    | —                 | Task priority level                                      |
| deadline                 | DATE           | nullable          | Task due date                                            |
| estimated_hours          | DECIMAL(10,2)  | ≥ 0               | Planned effort for progress weighting                    |
| total_time_spent_seconds | INTEGER        | ≥ 0               | Accumulated tracked time                                 |

### TimeLog — Individual work timer sessions

| Column           | Type           | Constraint | Description                                |
|------------------|----------------|------------|--------------------------------------------|
| task_id          | FK → Task      | CASCADE    | Associated task                            |
| user_id          | FK → auth_user | CASCADE    | Employee tracking time                     |
| start_time       | DATETIME       | —          | Session start                              |
| end_time         | DATETIME       | nullable   | NULL = active/running session              |
| last_activity_at | DATETIME       | —          | Last user activity timestamp               |
| duration_seconds | INTEGER        | —          | Computed on stop                           |
| source           | VARCHAR(20)    | —          | MANUAL_PAUSE   MANUAL_STOP   AUTO_STOP_8PM |

### Supporting Tables

| Table                        | Key Fields                                                               |
|------------------------------|--------------------------------------------------------------------------|
| Notification                 | user, type, title, message, ref_type, ref_id, is_read, details (JSON)    |
| FileAttachment               | project/milestone/task FK, uploaded_by, file, mime_type, size_bytes      |
| ProjectDeadlineChangeRequest | project, requested_by, current/new deadline, reason, status, reviewed_by |

### 3.3 Relationships & Constraints

- Unique constraint: (project, milestone\_no) — milestone numbers are unique per project
- Index: (project, status) on ProjectDeadlineChangeRequest for fast pending-request lookups
- Cascade deletes: Deleting a Project removes all Milestones, Tasks, and TimeLogs
- Cascade deletes: Deleting a Milestone removes all associated Tasks
- PROTECT on created\_by fields prevents accidental deletion of user records with created entities
- BigAutoField primary keys on all models (migration 0017)
- 24 database migrations track schema evolution from initial release to current version

## 4. Deployment

### 4.1 Environment Configuration

| Variable                       | Purpose                                               |
|--------------------------------|-------------------------------------------------------|
| DATABASE_URL                   | PostgreSQL connection string (required in production) |
| DJANGO_SECRET_KEY              | Cryptographic secret for Django sessions/signing      |
| DEBUG                          | False in production, True for local development       |
| ALLOWED_HOSTS                  | Comma-separated allowed hostnames/IPs                 |
| APP_PORT                       | Host port mapping (default: 6009)                     |
| CORS_ALLOW_ALL_ORIGINS         | CORS policy for frontend access                       |
| SMTP_USER / SMTP_PASS          | Email credentials for notifications                   |
| PMS_SERVICE_TOKEN              | Bearer token for internal service APIs                |
| OLLAMA_BASE_URL / OLLAMA_MODEL | AI assistant LLM endpoint and model                   |
| AUTO_STOP_CUTOFF_HOUR          | Timer auto-stop hour (default: 20 = 8 PM)             |

### 4.2 Local Development

#### Option A — Docker Compose (recommended):

```
cd pms
cp .env.example .env # Edit as needed
docker compose up --build
```

The dev container runs migrations, syncs bundled docs, ensures superuser, and starts Django runserver on port 6009 (mapped from container port 8000). SQLite is used when DATABASE\_URL is not set.

#### Option B — Direct Python:

```
cd pms
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

### 4.3 Docker Deployment

| File                    | Description                                                          |
|-------------------------|----------------------------------------------------------------------|
| Dockerfile              | Development image — Python 3.13-slim, runserver, volume mount        |
| Dockerfile.prod         | Production image — non-root user, Gunicorn (3 workers, 120s timeout) |
| docker-compose.yml      | Dev: build ., port 6009:8000, .env file, live code volume            |
| docker-compose.prod.yml | Prod: Dockerfile.prod, healthcheck, restart unless-stopped           |

**Production startup sequence:** migrate → sync\_bundled\_docs → ensure\_superuser → gunicorn pms.wsgi:application --bind 0.0.0.0:8000

## 4.4 Production (Jenkins CI/CD)

The Jenkins pipeline automates build, validation, and deployment to the production server.

| Stage                 | Action                                                           |
|-----------------------|------------------------------------------------------------------|
| 1. Checkout           | Pull latest code from SCM                                        |
| 2. Build Docker Image | docker build -f Dockerfile.prod → pms-backend:BUILD_NUMBER       |
| 3. Validate App       | docker run manage.py check with Jenkins credentials (.env)       |
| 4. Deploy             | Stop old container, run new image on port 6009 with media volume |
| 5. Verify Login       | Shell check: admin user exists, DB connection, auth test         |

### Production Endpoints:

- API Base: <https://nexus-pms.aspune.cloud/api/v1/>
- Swagger Docs: </api/docs/swagger/>
- ReDoc: </api/docs/redoc/>
- OpenAPI Schema: </api/schema/>
- Django Admin: </admin/>
- Frontend: <http://nexus.aspune.cloud>
- Server IP: 187.127.139.247 — Port: 6009

## 4.5 Scheduled Jobs & Integrations

| Management Command              | Purpose                                                         |
|---------------------------------|-----------------------------------------------------------------|
| send_deadline_notifications     | Weekday cron at 8 PM & 9 PM — auto-stop timers, deadline alerts |
| sync_project_milestone_statuses | Sync project/milestone statuses from task progress              |
| sync_bundled_docs               | Copy API_MASTER_GUIDE.md to media/project_docs/                 |
| ensure_superuser                | Create/promote admin user from env vars on startup              |

### External Integrations:

- **Attendance Service:** POST /api/v1/internal/notifications/ with PMS\_SERVICE\_TOKEN for leave notifications
- **Ollama AI:** Admin queries via /api/v1/admin/ai/ask with read-only database context
- **SMTP Email:** First-login OTP, password reset, deadline change notifications to admins
- **Nexus Frontend:** JWT-authenticated REST consumer on nexus.aspune.cloud

---

*Document generated automatically from the PMS codebase on June 11, 2026. For live API reference, visit /api/docs/swagger/ on the running server.*